LAPORAN PRAKTIKUM

STRUKTUR DATA

# "IMPLEMENTASI QUEUE MENGGUNAKAN ARRAY DAN LINKEDLIST"

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

# KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi Queue menggunakan Array dan LinkedList. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 28 April 2026

Hafizh Habibullah

# DAFTAR ISI

**KATA PENGANTAR** ..... 1
**DAFTAR ISI** ..... 2
**BAB I PENDAHULUAN** ..... 3
1.1 Latar Belakang ..... 3
1.2 Tujuan ..... 3
1.3 Manfaat ..... 3
**BAB 2 PEMBAHASAN** ..... 5
2.1 Implementasi Queue menggunakan Array ..... 5
2.1.1 Konstruktor dan Validator ..... 5
2.1.2 Enqueue dan Dequeue ..... 6
2.1.3 Front dan Rear ..... 7
2.1.4 Display ..... 8
2.2 Kelas Driver Queue Array ..... 9
2.3 Implementasi Queue menggunakan LinkedList ..... 10
2.4 Penggunaan Reverse Data ..... 11
2.5 Iterasi pada Queue ..... 12
2.6 Latihan Antrian Loket ..... 13
**BAB III KESIMPULAN** ..... 18
3.1 Kesimpulan ..... 18
**DAFTAR PUSTAKA** ..... 19

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pengembangan program karena berpengaruh terhadap efisiensi penyimpanan dan pengolahan data. Salah satu struktur yang sering digunakan adalah *queue* (antrian) yang menerapkan prinsip *First In First Out* (FIFO), di mana data yang pertama kali masuk akan menjadi data pertama yang keluar.

Implementasi *queue* dapat dilakukan dengan beberapa metode, di antaranya menggunakan *Array* dan *LinkedList*. Penggunaan *Array* relatif lebih sederhana, namun memiliki keterbatasan pada ukuran yang bersifat statis. Sementara itu, *Linked List* lebih fleksibel karena bersifat dinamis, tetapi memerlukan pengelolaan yang lebih kompleks.

Selain operasi dasar seperti *enqueue, dequeue,* dan *peek*, struktur *queue* juga dapat dikembangkan dengan operasi tambahan seperti *reverse* data. Oleh karena itu, diperlukan pemahaman mengenai berbagai metode implementasi *queue* untuk mengetahui kelebihan dan kekurangannya dalam pengolahan data.

## 1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data *queue* dengan prinsip FIFO.
2. Mengimplementasikan *queue* menggunakan array dan linked list.
3. Mengetahui perbedaan karakterisitk antara implementasi array dan linked list.
4. Mengaplikasikan operasi dasar seperti *enqueue, dequeue,* dan *peek*.
5. Mengimplementasikan operasi tambahan seperti *reverse* data.

## 1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Meningkatkan pemahaman mengenai struktur data *queue.*
2. Melatih kemampuan dalam mengimplementasikan struktur data menggunakan bahasa pemrograman Java.

3. Menambah wawasan mengenai kelebihan dan kekurangan array dan linked list.
4. Membantu dalam memahami pengolahan data secara lebih efisien.
5. Menjadi dasar dalam pengembangan program yang lebih kompleks.

# BAB 2
# PEMBAHASAN

## 2.1 Implementasi Queue menggunakan Array

### 2.1.1 Konstruktor dan Validator

```java
package pekan4_2511531002;

public class QueueArray_2511531002 {
    int front_1002, rear_1002, size_1002;
    int capacity_1002;
    int array_1002[];

    public QueueArray_2511531002(int capacity_1002) {
        this.capacity_1002 = capacity_1002;
        front_1002 = this.size_1002 = 0;
        rear_1002 = capacity_1002 - 1;
        array_1002 = new int[this.capacity_1002];
    }
```

Kode Program 2. 1 – Blok Program Konstruktor dan Validator

Class ini mengimplementasikan struktur data Queue menggunakan. Atribut front dan rear berfungsi sebagai penunjuk elemen terdepan dan terakhir, sedangkan size menyimpan jumlah elemen dalam antrian. Variabel capacity digunakan untuk menentukan kapasitas maksimum queue, dan array digunakan sebagai media penyimpanan data.

Pada konstruktor, nilai capacity diinisialisasi sesuai parameter yang diberikan. Atribut front dan size diatur ke 0 sebagai penanda bahwa antrian masih kosong, sedangkan rear diatur ke capacity - 1 untuk mendukung pergerakan indeks secara melingkar. Array kemudian dibuat sesuai dengan kapasitas yang telah ditentukan sehingga queue siap digunakan.

```java
    boolean isFull_1002(QueueArray_2511531002 queue) {
        return (queue.size_1002 == queue.capacity_1002);
    }

    boolean isEmpty_1002(QueueArray_2511531002 queue) {
        return (queue.size_1002 == 0);
    }
```

Kode Program 2. 2 – Blok Program Validator

Method isFull_1002 digunakan untuk memeriksa apakah queue sudah dalam keadaan penuh dengan cara membandingkan jumlah elemen saat ini (size_1002) dengan kapasitas maksimum queue (capacity_1002). Jika kedua nilai tersebut sama, maka queue dianggap penuh sehingga tidak dapat menampung elemen baru.

Method isEmpty_1002 digunakan untuk memeriksa apakah queue masih dalam keadaan kosong dengan cara mengecek apakah nilai size_1002 bernilai 0. Jika kondisi tersebut terpenuhi, maka queue belum memiliki elemen di dalamnya.

Kedua method ini berfungsi sebagai validator sebelum melakukan operasi pada queue, seperti penambahan (enqueue) maupun penghapusan (dequeue), agar tidak terjadi kesalahan akses data.

### 2.1.2 Enqueue dan Dequeue

```java
    void enqueue_1002(int item_1002) {
        if (isFull_1002(this))
            return;
        this.rear_1002 = (this.rear_1002 + 1) % this.capacity_1002;
        this.array_1002[this.rear_1002] = item_1002;
        this.size_1002 = this.size_1002 + 1;
        System.out.println(item_1002 + " enqueued to queue");
    }

    int dequeue_1002() {
        if (isEmpty_1002(this))
            return Integer.MIN_VALUE;
        int item_1002 = this.array_1002[this.front_1002];
        this.front_1002 = (this.front_1002 + 1) % this.capacity_1002;
        this.size_1002 = this.size_1002 - 1;
        return item_1002;
    }
```

Kode Program 2. 3 – Blok Program Penambahan dan Penghapusan

Method enqueue_1002 digunakan untuk menambahkan elemen ke dalam queue. Sebelum dilakukan, dicek terlebih dahulu apakah queue sudah penuh menggunakan isFull_1002. Jika belum penuh, nilai rear_1002 digeser secara melingkar menggunakan (rear_1002 + 1) % capacity_1002, lalu data dimasukkan ke array pada posisi tersebut. Lalu, size_1002 ditambah satu.

Method dequeue_1002 digunakan untuk menghapus elemen dari bagian depan queue. Sebelum dilakukan, dicek apakah queue kosong menggunakan isEmpty_1002. Jika tidak, elemen pada posisi front_1002 diambil, kemudian front_1002 digeser secara melingkar menggunakan (front_1002 + 1) % capacity_1002. Nilai size_1002 dikurangi satu, dan elemen yang dihapus dikembalikan sebagai output.

### 2.1.3 Front dan Rear

```
    int front_1002() {
        if (isEmpty_1002(this))
            return Integer.MIN_VALUE;
        return this.array_1002[this.front_1002];
    }
    int rear_1002() {
        if (isEmpty_1002(this))
            return Integer.MIN_VALUE;
        return this.array_1002[this.rear_1002];
    }
```

Kode Program 2. 4 – Blok Program Front dan Rear

Method front_1002() digunakan untuk mengambil nilai elemen terdepan pada queue dengan mengakses elemen pada indeks front_1002. Sebelum dilakukan pengambilan, sistem mengecek apakah queue kosong menggunakan isEmpty_1002. Jika kosong, method akan mengembalikan nilai Integer.MIN_VALUE sebagai penanda tidak ada data.

Sementara itu, method rear_1002() digunakan untuk mengambil nilai elemen terakhir pada queue dengan mengakses elemen pada indeks rear_1002. Sama seperti method sebelumnya, pengecekan kondisi kosong

dilakukan terlebih dahulu, dan jika queue kosong maka akan mengembalikan Integer.MIN_VALUE.

### 2.1.4 Display

```
    void display_1002() {
        int i;
        if (front_1002 == rear_1002) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        for (i = front_1002; i < rear_1002; i++) {
            System.out.printf(" %d ←", array_1002[i]);
        }
        return;
    }
}
```

Kode Program 2. 5 – Blok Program untuk Menampilkan elemen queue

Class ini berisi method display_1002() yang digunakan untuk menampilkan seluruh elemen yang terdapat dalam queue. Pada awalnya, dilakukan pengecekan apakah queue kosong dengan membandingkan nilai front_1002 dan rear_1002. Jika keduanya sama, maka queue akan dianggap kosong dan program akan menampilkan pesan “Antrian Kosong”.

Jika tidak kosong, maka dilakukan perulangan dari indeks front_1002 hingga rear_1002 – 1 untuk menampilkan seluruh elemen yang ada di dalam array. Setiap elemen ditampilkan secara berurutan dengan format panah (←) untuk menggambarkan urutan antrian.

## 2.2 Kelas Driver Queue Array

```java
package pekan4_2511531002;

public class QueueArrayDriver_2511531002 {
    public static void main(String[] args) {
        QueueArray_2511531002 queue_1002 = new QueueArray_2511531002(1000);
        queue_1002.enqueue_1002(10);
        queue_1002.enqueue_1002(20);
        queue_1002.enqueue_1002(30);
        queue_1002.enqueue_1002(40);
        System.out.println("Item di depan: " + queue_1002.front_1002());
        System.out.println("Item paling belakang: " + queue_1002.rear_1002());
        System.out.println("Tampilan Queue");
        queue_1002.display_1002();
        System.out.println();
        System.out.println(queue_1002.dequeue_1002() + " dihapus dari queue");
        System.out.println("Item di depan: " + queue_1002.front_1002());
        System.out.println("Item di belakang: " + queue_1002.rear_1002());
        System.out.println("Tampilan queue setelah satu data dihapus");
        queue_1002.display_1002();
    }
}
```

Kode Program 2. 6 – Program Driver untuk Stack Array

Program ini digunakan untuk menguji implementasi Queue berbasis array. Data dimasukkan ke queue menggunakan method enqueue_1002, kemudian ditampilkan elemen depan, belakang, dan isi queue. Setelah itu dilakukan penghapusan data dengan dequeue_1002, sehingga elemen paling depan dihapus. Program kemudian kembali menampilkan kondisi queue setelah perubahan. Secara keseluruhan, program ini menunjukkan operasi dasar queue yaitu enqueue, dequeue, serta akses elemen depan dan belakang.. Hasil Outputnya adalah sebagai berikut:

```
Console   Progress   Git Staging
<terminated> QueueArrayDriver_2511531002 [Java Application] C:\Us
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan: 10
Item paling belakang: 40
Tampilan Queue
 10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan: 20
Item di belakang: 40
Tampilan queue setelah satu data dihapus
 20 <-- 30 <--
```

Gambar 2. 1 – Hasil Output dari Program Driver Queue Array

## 2.3 Implementasi Queue menggunakan LinkedList

```java
package pekan4_2511531002;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511531002 {
    public static void main(String[] args) {
        Queue<Integer> q_1002 = new LinkedList<>();
        // tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
        for (int i = 0; i < 6; i++)
            q_1002.add(i);
        // Menampilkan isi antrian
        System.out.println("Elemen Antrian " + q_1002);
        // Untuk menghapus kepala antrian
        int hapus_1002 = q_1002.remove();
        System.out.println("Hapus elemen = " + hapus_1002);
        System.out.println(q_1002);
        // Untuk melihat antrian terdepan
        int depan_1002 = q_1002.peek();
        System.out.println("Kepala Antrian = " + depan_1002);

        int banyak_1002 = q_1002.size();
        System.out.println("Size Antrian = " + banyak_1002);
    }
}
```

Kode Program 2. 7 – Program Queue LinkedList

Program ini merupakan implementasi Queue menggunakan **LinkedList** dari library Java (java.util.Queue). Pada bagian awal, dibuat objek queue q_1002 yang bertipe Integer dengan menggunakan struktur LinkedList.

Selanjutnya, program menambahkan elemen ke dalam queue menggunakan perulangan for dari 0 sampai 5. Setiap nilai ditambahkan ke antrian dengan method add(), sehingga queue berisi elemen secara berurutan sesuai konsep FIFO (First In First Out).

Setelah itu, isi queue ditampilkan untuk menunjukkan kondisi awal antrian. Kemudian dilakukan operasi penghapusan menggunakan method remove(), yang secara otomatis menghapus elemen paling depan (head queue). Nilai yang dihapus disimpan dalam variabel hapus_1002 dan ditampilkan ke layar.

Program kemudian menampilkan kembali isi queue setelah penghapusan untuk menunjukkan perubahan data. Selanjutnya, method peek() digunakan untuk melihat elemen terdepan tanpa menghapusnya, yang disimpan dalam variabel depan_1002.

Terakhir, program menampilkan ukuran queue menggunakan method size() untuk mengetahui jumlah elemen yang masih tersisa dalam antrian. Hasil programnya adalah sebagai berikut:

```
Console ×   Progress   Git Staging
<terminated> QueueLinkedList_2511531002 [Java
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2. 2 – Hasil Program Queue Linked List

## 2.4 Penggunaan Reverse Data

```java
package pekan4_2511531002;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511531002 {
    public static void main(String[] args) {
        Queue<Integer> q_1002 = new LinkedList<Integer>();
        q_1002.add(1);
        q_1002.add(2);
        q_1002.add(3);
        System.out.println("Sebelum reverse = " + q_1002);
        Stack<Integer> s_1002 = new Stack<Integer>();
        while (!q_1002.isEmpty()) {
            s_1002.push(q_1002.remove());
        }
        while (!s_1002.isEmpty()) {
            q_1002.add(s_1002.pop());
        }
        System.out.println("Sesudah reverse = " + q_1002);
    }
}
```

Kode Program 2. 8 – Program Reverse Data dengan Queue

Program ini digunakan untuk melakukan proses **membalik urutan data (reverse)** pada struktur Queue dengan bantuan Stack. Awalnya dibuat sebuah Queue bertipe Integer menggunakan LinkedList, kemudian dimasukkan tiga data yaitu 1, 2, dan 3. Isi queue sebelum proses reverse ditampilkan untuk menunjukkan urutan awal data sesuai konsep FIFO (First In First Out).

Selanjutnya dibuat Stack sebagai struktur bantuan. Program kemudian memindahkan semua elemen dari queue ke stack menggunakan perulangan while, dengan cara menghapus elemen depan queue (remove()) lalu memasukkannya ke stack (push()). Karena Stack bersifat LIFO (Last In First Out), urutan data otomatis terbalik saat dipindahkan.

Setelah itu, data dipindahkan kembali dari stack ke queue menggunakan pop() dan add(), sehingga queue kembali terisi tetapi dengan urutan yang sudah dibalik. Terakhir, isi queue ditampilkan kembali untuk menunjukkan hasil setelah proses reverse, yang dimana hasil output dari contoh program reverse data adalah sebagai berikut:

```
Console   Progress   Git St
<terminated> ReverseData_2511531002 [
Sebelum reverse = [1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

Gambar 2. 3 – Hasil Program Reverse Data Queue

## 2.5 Iterasi pada Queue

```java
package pekan4_2511531002;

import java.util.Queue;
import java.util.LinkedList;
import java.util.Iterator;

public class IterasiQueue_2511531002 {
    public static void main(String[] args) {
        Queue<String> q_1002 = new LinkedList<>();

        q_1002.add("Praktikum");
        q_1002.add("Struktur");
        q_1002.add("Data");
        q_1002.add("Dan");
        q_1002.add("Algoritma");
        Iterator<String> iterator = q_1002.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }
}
```

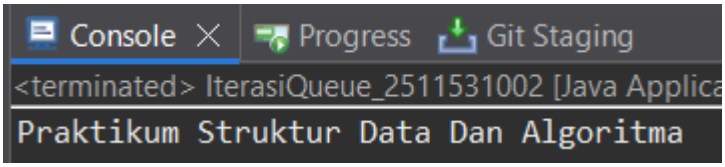
Kode Program 2. 9 – Penggunaan Iterasi pada Queue

Program ini digunakan untuk melakukan **iterasi (perulangan) pada Queue** menggunakan Iterator.

Pada awalnya dibuat sebuah Queue bertipe String menggunakan LinkedList, kemudian dimasukkan beberapa data yaitu “Praktikum”, “Struktur”, “Data”, “Dan”, dan “Algoritma”. Data tersebut tersimpan di dalam queue sesuai urutan penambahan (FIFO).

Selanjutnya, dibuat sebuah Iterator untuk membaca isi queue tanpa menghapus data di dalamnya. Dengan menggunakan perulangan while (iterator.hasNext()), setiap elemen queue diakses satu per satu menggunakan iterator.next() dan langsung ditampilkan ke layar.

Berbeda dengan operasi remove() atau dequeue(), penggunaan iterator ini hanya berfungsi untuk membaca isi queue tanpa memodifikasi data yang ada. Hasil output dari Program ini adalah sebagai berikut:




Gambar 2. 4 – Hasil Program Iterasi pada Queue

## 2.6 Latihan Antrian Loket

```
package pekan4_2511531002;

public class Queue_2511531002 {
	int front_1002, rear_1002;
	int max_1002;
	String queue_1002[];

	public Queue_2511531002(int size_1002) {
		max_1002 = size_1002;
		queue_1002 = new String[max_1002];
		front_1002 = 0;
		rear_1002 = 0;
	}

	boolean isFull_1002() {
		return rear_1002 == max_1002;
	}
```

```
	boolean isEmpty_1002() {
		return front_1002 == rear_1002;
	}

	void enqueue_1002(String name_1002) {
		if (isFull_1002()) {
			System.out.println("Antrian penuh");
		} else {
			queue_1002[rear_1002] = name_1002;
			rear_1002++;
			System.out.println("Data berhasil ditambahkan
ke antrian");
		}
	}

	void dequeue_1002() {
		if (isEmpty_1002()) {
			System.out.println("Antrian kosong");
		} else {
			System.out.println(queue_1002[front_1002] + "
telah dilayani");
			front_1002++;
		}
	}

	void display_1002() {
		if (isEmpty_1002()) {
			System.out.println("Antrian kosong");
		} else {
			System.out.println("Isi antrian:");
			int no_1002 = 1;
			for (int i_1002 = front_1002; i_1002 <
rear_1002; i_1002++) {
				System.out.println(no_1002 + ". " +
queue_1002[i_1002]);
				no_1002++;
			}
		}
	}

	void reverse_1002() {
		if (isEmpty_1002()) {
			System.out.println("Antrian kosong");
		} else {
			int start_1002 = front_1002;
			int end_1002 = rear_1002 - 1;

			while (start_1002 < end_1002) {
				String temp_1002 =
queue_1002[start_1002];
				queue_1002[start_1002] =
queue_1002[end_1002];
				queue_1002[end_1002] = temp_1002;

				start_1002++;
				end_1002--;
			}
```

```
                System.out.println("Antrian berhasil dibalik");
            }
        }

}
```

Kode Program 2.10 – Tugas Queue_2511531002

Class ini digunakan untuk mengimplementasi struktur data queue menggunakan array, dengan atribut front_1002 sebagai penunjuk depan, rear_1002 sebagai penunjuk belakang, max_1002 sebagai kapasitas, dan queue_1002[] sebagai tempat menyimpan data. Konstruktor digunakan untuk menginisialisasi ukuran array serta mengatur nilai awal front_1002 dan rear_1002 menjadi 0.

Method isFull_1002() berfungsi mengecek apakah antrian penuh (rear_1002 == max_1002), sedangkan isEmpty_1002() mengecek apakah antrian kosong (front_1002 == rear_1002). Method enqueue_1002() menambahkan data ke belakang antrian jika belum penuh, sementara dequeue_1002() menghapus data dari depan jika tidak kosong.

Method display_1002() digunakan untuk menampilkan isi antrian dari depan ke belakang, dan reverse_1002() berfungsi membalik urutan elemen dalam antrian dengan menukarkan posisi elemen dari depan dan belakang hingga ke tengah.

```
package pekan4_2511531002;

import java.util.Scanner;

public class AntrianLoket_2511531002 {
    public static void main(String[] args) {
        Scanner input_1002 = new Scanner(System.in);
        Queue_2511531002 q_1002 = new Queue_2511531002(10);

        int pilihan_1002;
        String nama_1002;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET
===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
```

```
                    pilihan_1002 = input_1002.nextInt();
                    input_1002.nextLine();

                    switch (pilihan_1002) {
                    case 1:
                          System.out.println("Masukkan nama
pelanggan: ");
                          nama_1002 = input_1002.nextLine();
                          q_1002.enqueue_1002(nama_1002);
                          break;
                    case 2:
                          q_1002.dequeue_1002();
                          break;
                    case 3:
                          q_1002.display_1002();
                          break;
                    case 4:
                          q_1002.reverse_1002();
                          break;
                    case 5:
                          System.out.println("Program selesai");
                          break;
                    default:
                          System.out.println("Pilihan tidak
valid");
                    }
              } while (pilihan_1002 != 5);
              input_1002.close();
        }
}
```

Kode Program 2.11 – Tugas AntrianLoket_2511531002

Program AntrianLoket_2511531002 merupakan program utama yang digunakan untuk menjalankan simulasi antrian loket menggunakan class sebelumnya, yaitu Queue_2511531002. Program ini memanfaatkan Scanner untuk menerima input dari pengguna dan membuat objek queue dengan kapasitas 10 data. Variabel pilihan_1002 digunakan untuk menyimpan pilihan menu, sedangkan nama_1002 untuk menyimpan input nama pelanggan.

Program berjalan menggunakan perulangan do-while yang akan terus menampilkan menu hingga pengguna memilih keluar. Menu yang tersedia meliputi penambahan antrian, penghapusan antrian, menampilkan isi antrian, serta membalik urutan antrian. Struktur switch-case digunakan untuk mengeksekusi pilihan pengguna, seperti memanggil method enqueue_1002(), dequeue_1002(), display_1002(), dan reverse_1002() sesuai menu yang dipilih. Program akan

berhenti ketika pengguna memilih opsi keluar, kemudian Scanner ditutup. Hasil output program akan terlihat seperti berikut:

```
Console  Progress  Git Staging
AntrianLoket_2511531002 [Java Application] C:\Users\user\.p2

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan:
Rizal Ekonomi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan:
Fuad Kicau Mania
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Rizal Ekonomi
2. Fuad Kicau Mania
```

```
Console  Progress  Git Staging
AntrianLoket_2511531002 [Java Application] C:\Users\u

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Rizal Ekonomi
2. Fuad Kicau Mania

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Fuad Kicau Mania
2. Rizal Ekonomi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Fuad Kicau Mania telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

Gambar 2. 5 – Hasil Program Antrian Loket

# BAB III
# KESIMPULAN

## 3.1 Kesimpulan

Dari hasil praktikum Implementasi Queue menggunakan Array dan LinkedList, dapat disimpulkan bahwa queue merupakan struktur data dengan prinsip FIFO (First In First Out), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

Implementasi queue menggunakan array memiliki kelebihan dari sisi kesederhanaan dalam pengelolaan data, namun memiliki keterbatasan pada ukuran yang bersifat statis sehingga tidak fleksibel ketika jumlah data berubah-ubah. Sementara itu, implementasi menggunakan LinkedList lebih fleksibel karena bersifat dinamis, sehingga dapat menyesuaikan ukuran sesuai kebutuhan, meskipun proses pengelolaannya sedikit lebih kompleks dibanding array.

Selain operasi dasar seperti enqueue, dequeue, dan peek, queue juga dapat dikombinasikan dengan struktur data lain seperti stack untuk melakukan operasi tambahan seperti reverse data. Selain itu, penggunaan iterator memungkinkan proses penelusuran isi queue tanpa mengubah data yang ada di dalamnya.

Secara keseluruhan, praktikum ini membantu dalam memahami cara kerja queue serta perbedaan implementasinya menggunakan array dan LinkedList, sekaligus memberikan gambaran bagaimana struktur data dapat digunakan secara efektif dalam pemrograman.

## DAFTAR PUSTAKA

[1] Modul Praktikum Struktur Data.